Forward-Mode Automatic Differentiation

CMSC22300 Final Project

Jay Shen

What is Automatic Differentiation?

- **Symbolic**: manipulates expressions $\Rightarrow$ expression swell
- **Numerical**: finite differences $\Rightarrow$ approximation error
- **Automatic**: exact derivatives via operator overloading

This project implements forward-mode AD in Haskell using **dual numbers**.

Dual Numbers

A dual number carries a value and its derivative:

```
data Dual a = Dual { primal :: a, tangent :: a }
```

Arithmetic encodes differentiation rules, e.g. the product rule:

$$(x, x') \cdot (y, y') = (xy,\ x'y + xy')$$

`Num`, `Fractional`, and `Floating` instances encode the chain rule for all elementary operations.

Differentiation Combinators

```
var x       = Dual x 1
diff f      = \x -> tangent (f (var x))
grad f xs   = [partial derivatives]
```

`diff` returns a first-class function, so higher-order derivatives come from composition:

- `diff f` $= f'$
- `(diff . diff) f` $= f''$
- `(diff . diff . diff) f` $= f'''$

Linear Regression

Minimize MSE over $n = 50$ noisy samples from $y = mx + b$:

$$L(m, b) = \frac{1}{n} \sum_{i=1}^{n} (y_i - (mx_i + b))^2$$

Gradient descent with $\eta = 0.001$, 1000 steps:

Param	**True**	**Fitted**	**Loss**
m	3.0	3.029	0.212
b	1.0	0.874	

Logistic Regression

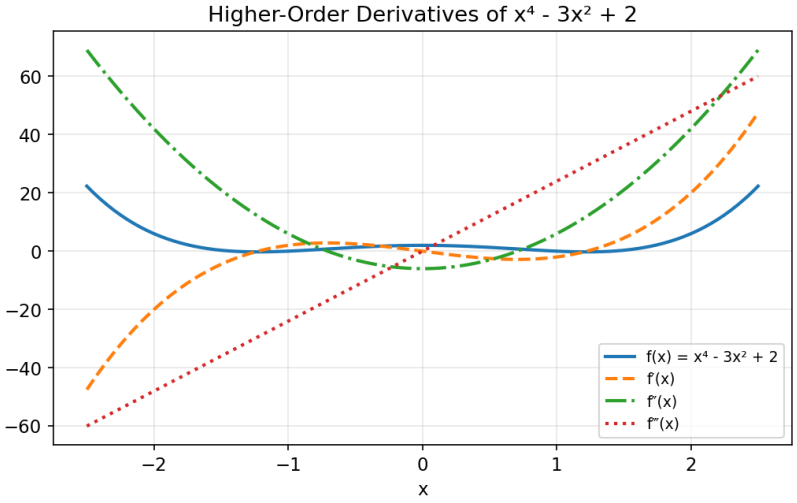
Binary classification with cross-entropy loss:

$$L(w, b) = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]$$

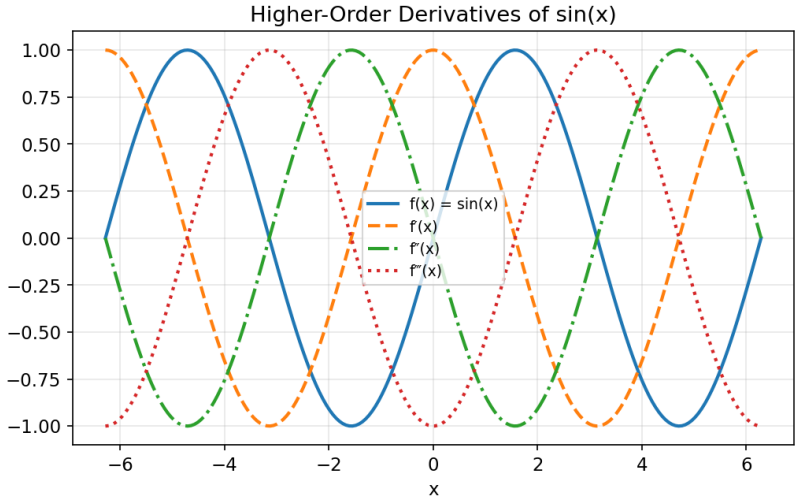
Gradient descent with $\eta = 0.1$, 1000 steps:

Param	**Fitted**	**Loss**	**Accuracy**
w	3.360	0.032	99%
b	0.106		

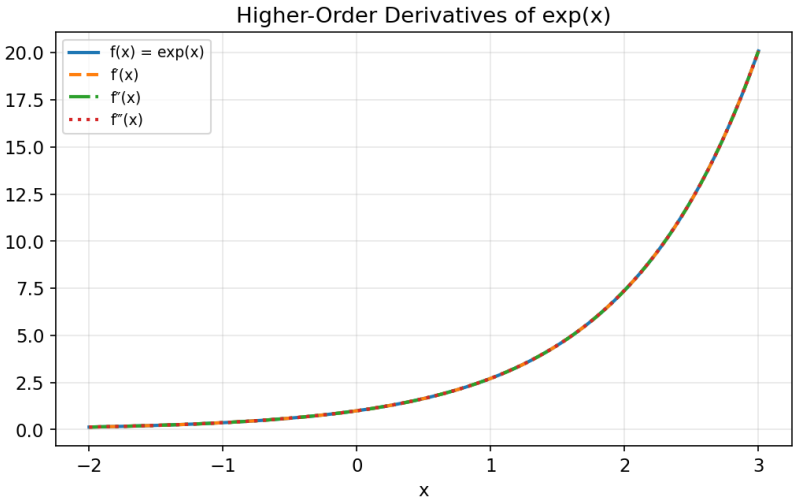
Higher-Order Derivatives



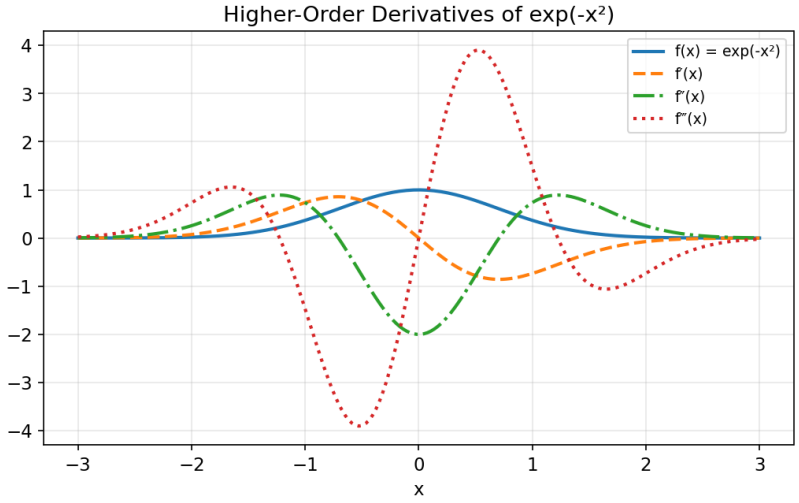
(a) $x^4 - 3x^2 + 2$



(b) $\sin(x)$



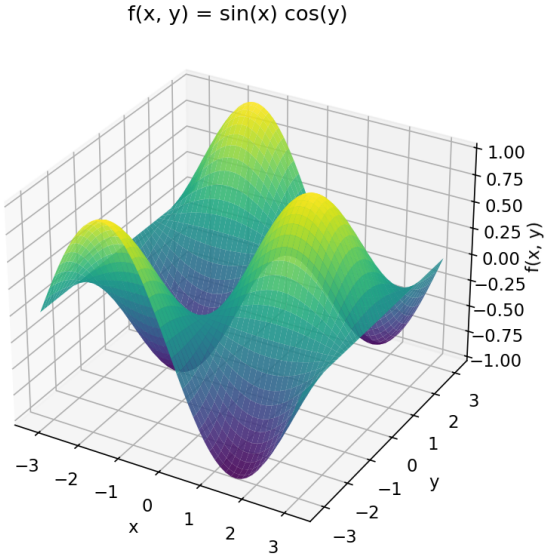
(c) e^x

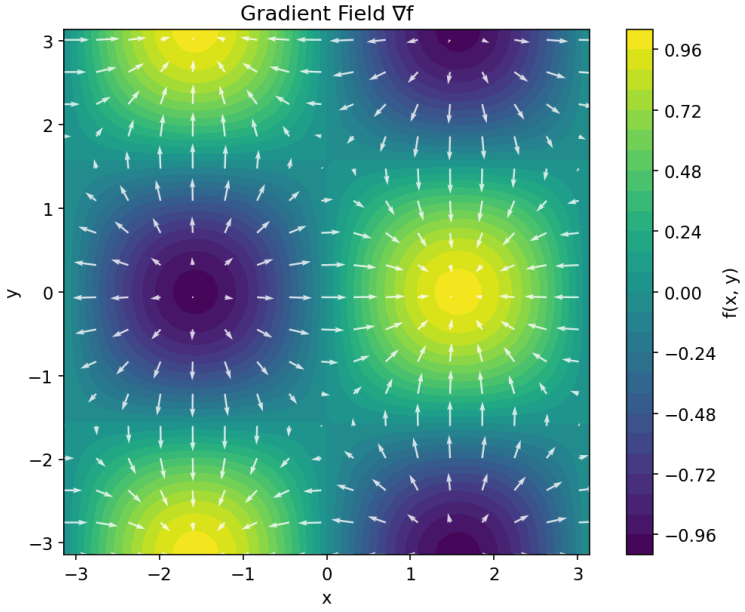


(d) e^{-x^2}

Gradient Field

$f(x, y) = \sin(x)\cos(y)$ over $[-\pi, \pi]^2$:





Summary

- Forward-mode AD via dual numbers gives exact derivatives
- Haskell's typeclass system makes the implementation concise
- Higher-order derivatives via simple function composition
- Gradients accurate enough to drive gradient descent to convergence